Software Engineering & System Architecture

Solved Assignments 1, 2, 3 & 4 with Comprehensive UML Project Modeling

Student Name: Aditya Pawar	**Course / Subject:** Software Engineering & Testing
Project: AI-Powered Mental Health & Well-being Companion	**Technology Stack:** Python (Flask) / PHP, MySQL, OpenAI API

ASSIGNMENT 1: PROJECT DETAILS

1. Describe your project.

The project is an **AI-Powered Mental Health & Well-being Companion** (Mood & Journal Tracker). It is an interactive, privacy-focused web platform designed to support mental wellness through daily mood logging, written and voice journaling, standardized psychological questionnaires (stress, anxiety, happiness, loneliness), daily motivational quotes, real-time conversational AI emotional support (powered by OpenAI API), a directory of nearby verified therapists (via Google Maps API), and immediate SOS emergency crisis contact access.

2. Advantages of your project.

- **24/7 Accessibility:** Instant, confidential emotional support and guided journaling available around the clock.
- **Multimodal Journaling:** Accommodates both text reflections and audio voice recordings for flexible self-expression.
- **Crisis Readiness:** Provides direct one-tap access to local emergency hotlines and nearby mental health clinics.
- **Actionable Visual Analytics:** Generates trend charts that help users recognize emotional triggers and progress over time.

3. How do you gather the requirements of your project?

- **Psychological Literature & Clinical Scales:** Researched standardized psychological assessment tools including PHQ-9 (depression) and GAD-7 (anxiety).
- **User Surveys & Field Interviews:** Surveyed individuals regarding emotional tracking habits, mental health stigma, and data privacy expectations.
- **Competitor & App Benchmarking:** Analyzed features of leading applications like Woebot, Moodfit, and Daylio.
- **Counselor Consultations:** Engaged with student counselors to establish safe boundaries for AI-generated guidance.

4. How and where do you implement your project?

- **Development Environment:** Developed locally on VS Code using a local Apache/Flask web server and MySQL database server.
- **Implementation Architecture:** Built following a clean 3-Tier architecture: **Presentation Layer** (HTML5, CSS3, JavaScript), **Application Layer** (Python Flask / PHP REST API endpoints), and **Data Layer** (MySQL relational database), interfaced with external cloud APIs.

5. Which software you used in your project?

- **IDE & Tools:** Visual Studio Code, Git, Postman API Tester, phpMyAdmin.
- **Frontend Technologies:** HTML5, CSS3, JavaScript (ES6+), Tailwind CSS.
- **Backend Environment:** Python 3.10+ (Flask framework) / PHP 8.x.
- **Database:** MySQL Community Server 8.0.
- **Third-Party Cloud APIs:** OpenAI API (Chat Completions) and Google Maps Platform API (Places & Geolocation).

6. What methodology is used for the project?

Agile (Scrum) Methodology: The project was developed across two-week iterative sprints: Sprint 1 (User Auth & Database Schema), Sprint 2 (Mood & Journal Modules), Sprint 3 (Voice Recording & OpenAI Chatbot), and Sprint 4 (Therapist Locator & Emergency SOS). This approach ensured continuous validation and rapid feature delivery.

7. What is the hardware or software of the project?

- **Hardware Requirements:**
  - *Server/Developer:* Multi-core processor (Intel Core i3/i5 or Apple Silicon), 8 GB+ RAM, 256 GB SSD, microphone for voice notes.
  - *Client:* Any desktop, laptop, tablet, or smartphone with a microphone and internet connection.
- **Software Requirements:** Windows 10/11, macOS, or Linux OS; modern web browser (Google Chrome, Edge, Safari); Python 3.10+ / PHP 8; MySQL 8.0.

ASSIGNMENT 1: PROJECT DETAILS (CONTINUED)

8. What is your role in the project?

Full-Stack Developer & System Architect: Responsible for the complete end-to-end delivery: gathering requirements, designing UML diagrams and MySQL relational schemas, developing responsive web interfaces, coding backend business logic and REST endpoints, integrating OpenAI and Google Maps APIs, and performing functional test validation.

9. Explain the data flow of your application.

1. **User Authentication:** User submits login credentials $\rightarrow$ Web App forwards payload to Server $\rightarrow$ Server verifies password hash in MySQL $\rightarrow$ User session token generated and returned.
2. **Mood & Journal Logging:** User enters mood score, written reflection, or voice note $\rightarrow$ HTTP POST to Server $\rightarrow$ Database record inserted $\rightarrow$ Dashboard analytics engine updates visual charts.
3. **AI Companion Conversation:** User sends message in AI chat $\rightarrow$ Server forwards message with conversational context to OpenAI API $\rightarrow$ AI generates empathetic response $\rightarrow$ Returned and rendered in chat view.
4. **Emergency Support Flow:** User triggers Emergency Help $\rightarrow$ Web App immediately presents emergency hotlines and calls Google Maps API with user coordinates to display nearby clinics.

10. What are the entities in your project?

- User: userId, name, email, password (Signup and authentication)
- Mood: moodId, userId, mood, date, time (Daily mood levels and timestamps)
- Journal: journalId, userId, text, date (Reflective journal entries)
- VoiceNote: voiceId, userId, audioPath, date, duration (Stored audio notes)
- Questionnaire: questionId, userId, stressScore, anxietyScore, happinessScore, lonelinessScore, date
- Quote: quoteId, quote, author (Inspirational daily quotes)
- AI_Friend: Methods for chat(), suggestTips(), breathingGuide(), journalingPrompts()
- Therapist: therapistId, name, phone, address, latitude, longitude (Local practitioner directory)
- EmergencyContact: id, name, phone, type (SOS helpline numbers)

11. What is the drawback of your system?

- **Not a Medical Diagnostic Tool:** Cannot replace certified clinical psychiatric evaluation, therapy, or medicinal prescription.
- **Connectivity & API Dependence:** AI conversational capabilities and map lookups require continuous internet connectivity and external API uptime.
- **Self-Reporting Bias:** Accuracy of analytical insights relies on consistent, truthful user self-reporting.

12. How do you test your application?

- **Unit Testing:** Verified individual backend logic (e.g., authentication hashing, questionnaire score aggregations, input sanitization).
- **Integration Testing:** Tested end-to-end data pipelines connecting frontend web forms, backend endpoints, MySQL, and OpenAI API.
- **Black-Box & UI Testing:** Validated form validations, audio microphone capture, and cross-browser responsiveness across screen sizes.
- **User Acceptance Testing (UAT):** Conducted trials with sample users to assess UI clarity, conversational empathy, and navigation.

13. From where do you get reference, guide for your project work?

- Academic research articles on cognitive behavioral therapy (CBT) and mobile mental health informatics.
- Official API and developer documentation: OpenAI Platform docs, Python Flask / PHP manuals, MySQL 8.0 Reference, MDN Web Docs.
- Academic project mentor and faculty guidelines, alongside peer reviews.

14. In the future, what changes will you make in your application?

- **Acoustic Emotion Recognition:** Implement machine learning models to detect emotional distress directly from voice note audio tone.
- **Smartwatch/Wearable Sync:** Integrate with smart bands to correlate physiological data (heart rate, HRV, sleep) with logged moods.
- **Teletherapy Telehealth Module:** Introduce secure, end-to-end encrypted video appointments with certified psychologists.
- **Offline Mode Support:** Convert to a Progressive Web App (PWA) with local storage and automatic background synchronization.

ASSIGNMENT 2: UML (UNIFIED MODELING LANGUAGE)

1. What is UML?

Unified Modeling Language (UML) is a standardized, general-purpose visual modeling language in software engineering managed by the Object Management Group (OMG). It offers a standard graphical vocabulary to visualize, specify, construct, and document the structure and behavior of object-oriented software systems.

2. What is the use of UML?

• **Visual Blueprinting:** Enables system architects and developers to conceptualize software models and database relationships before writing code.
• **Cross-Functional Communication:** Creates an unambiguous common language understood by developers, testers, analysts, and project sponsors.
• **Early Flaw Detection:** Reveals architectural bottlenecks, unhandled use cases, and logic gaps in early development phases.
• **System Documentation & Maintainability:** Serves as reliable documentation that eases maintenance, refactoring, and team onboarding.

3. Explain types of UML diagram with example?

UML diagrams are divided into two fundamental classifications:
1. **Structural Diagrams (Static Modeling):** Represent the static building blocks of a system that remain fixed over time.
 – *Class Diagram:* Displays classes, attributes, methods, and relationships (inheritance, association).
 – *Component Diagram:* Depicts organizational relationships between physical software modules, components, and APIs.
 – *Deployment Diagram:* Maps software components to physical hardware execution nodes and networks.
2. **Behavioral Diagrams (Dynamic Modeling):** Depict the dynamic flow of control, state transitions, and interactions over time.
 – *Use Case Diagram:* Maps interactions between human/external actors and system functions inside system boundaries.
 – *Sequence Diagram:* Captures chronological time-ordered message passing between interacting objects.
 – *Activity Diagram:* Models procedural workflows, decision logic, and parallel operations like a flowchart.

4. Draw UML Diagrams of your project.

Figure 2.0 provides the consolidated master reference architecture containing all 5 UML diagrams (Use Case, Class, Activity, Sequence, and Component) created for the **AI Mental Health & Well-being Companion**. Individual diagrams are detailed on subsequent pages.

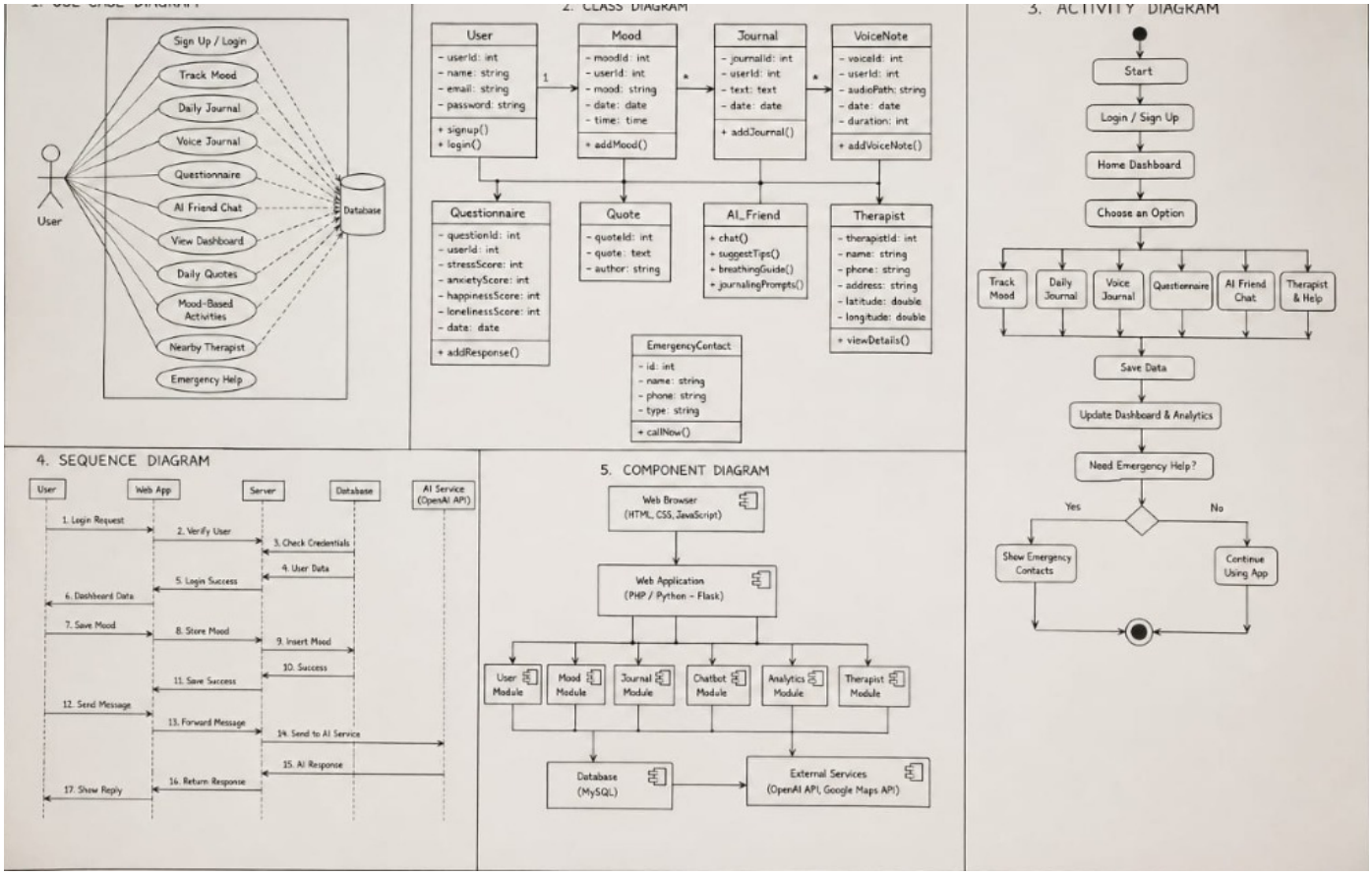


Figure 2.0: Master Architectural UML Sheet (Consolidated System Models)

ASSIGNMENT 2: UML MODELING — 4.1 USE CASE DIAGRAM

4.1 Use Case Diagram Overview & Analysis

The Use Case Diagram captures the interactions between the primary actor (User) and system services within the *Mental Health & Well-being System* boundary. Each use case communicates with the central Database for storage and retrieval.

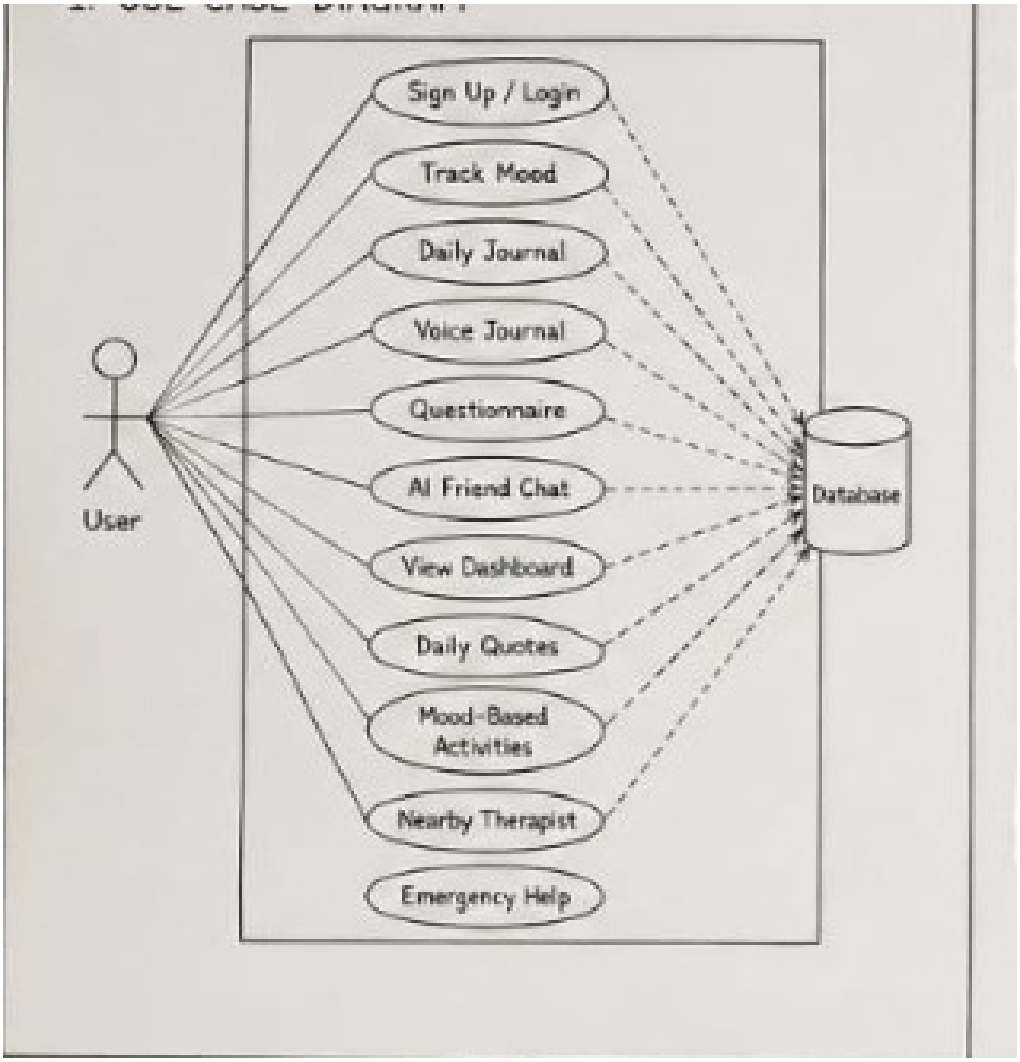


Figure 2.1: Use Case Diagram of the Mental Health & Well-being Application

Actor	Key Use Cases Handled	Backend Integration
User (Client / End-User)	• Sign Up / Login • Track Mood & Daily Journal • Voice Journal & Questionnaire • AI Friend Chat & View Dashboard • Daily Quotes & Mood-Based Activities • Nearby Therapist & Emergency Help	All user inputs are authenticated, validated, and persisted into the relational database. AI Chat routes through OpenAI API, and Therapist Search queries Google Maps API.

ASSIGNMENT 2: UML MODELING — 4.2 CLASS DIAGRAM

4.2 Class Diagram Overview & Entity Relationships

The Class Diagram illustrates the object-oriented structure of the software. It specifies class attributes, methods, visibility access modifiers, and cardinality multiplicity (e.g., 1 User has * Moods, * Journals, * VoiceNotes, and * Questionnaires).

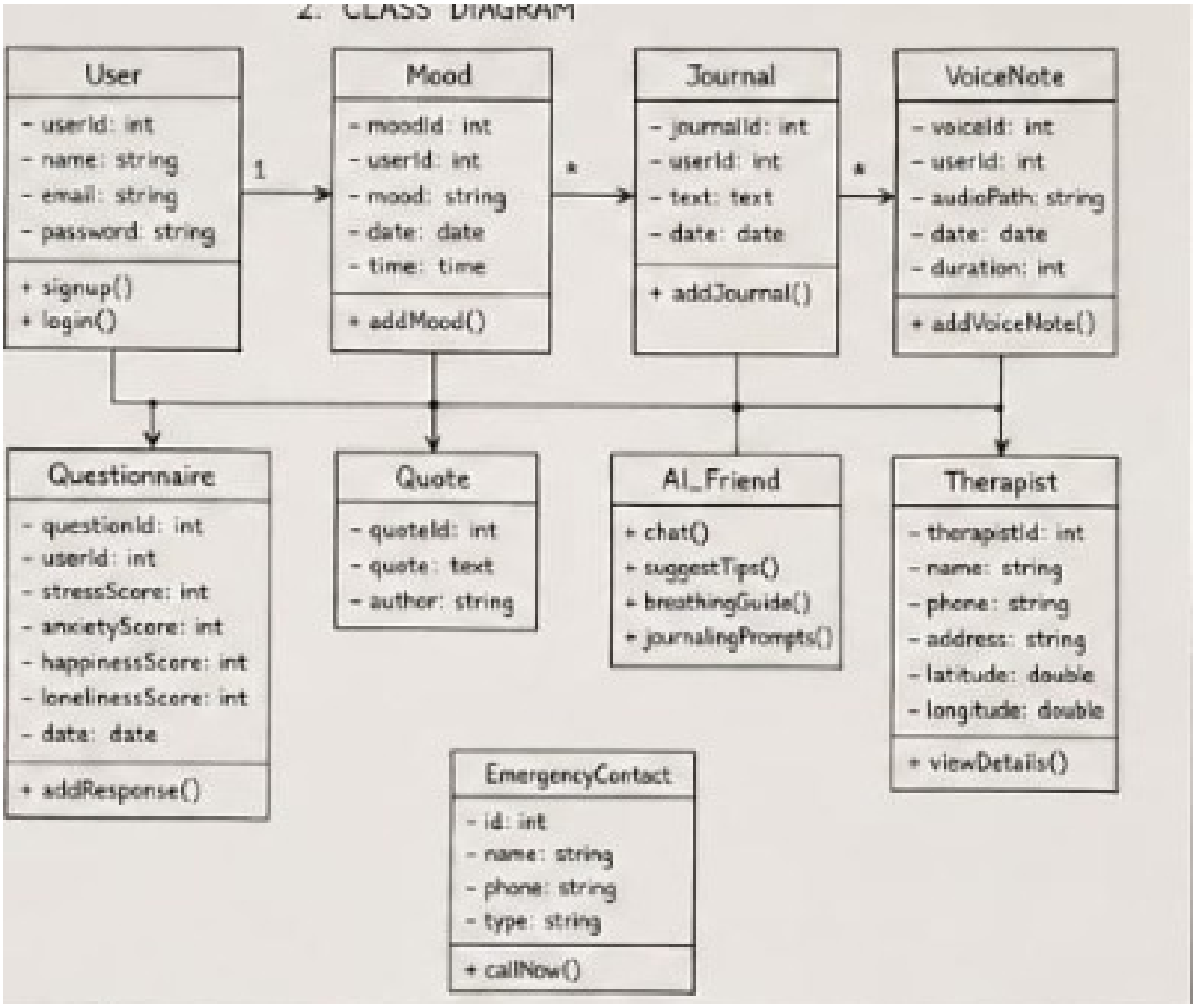


Figure 2.2: Class Diagram showing Attributes, Methods, and Associations

Class Name	Key Attributes	Primary Methods & Multiplicity
User	userId, name, email, password	+ signup(), + login() (1 to many with tracking records)
Mood / Journal / VoiceNote	moodId, journalId, voiceId, date, time, text, audioPath, duration	+ addMood(), + addJournal(), + addVoiceNote()
Questionnaire	questionId, stressScore, anxietyScore, happinessScore, lonelinessScore	+ addResponse() (Stores normalized wellness assessment scores)
AI_Friend / Services	quoteId, therapistId, latitude, longitude, phone, contact id	+ chat(), + suggestTips(), + breathingGuide(), + viewDetails(), + callNow()

ASSIGNMENT 2: UML MODELING — 4.3 ACTIVITY DIAGRAM

4.3 Activity Diagram Workflow & Logic

The Activity Diagram models the operational control flow: Start $\rightarrow$ Authentication $\rightarrow$ Dashboard $\rightarrow$ User selects an activity $\rightarrow$ System saves data $\rightarrow$ Analytics are refreshed $\rightarrow$ Condition check for crisis support $\rightarrow$ Termination.

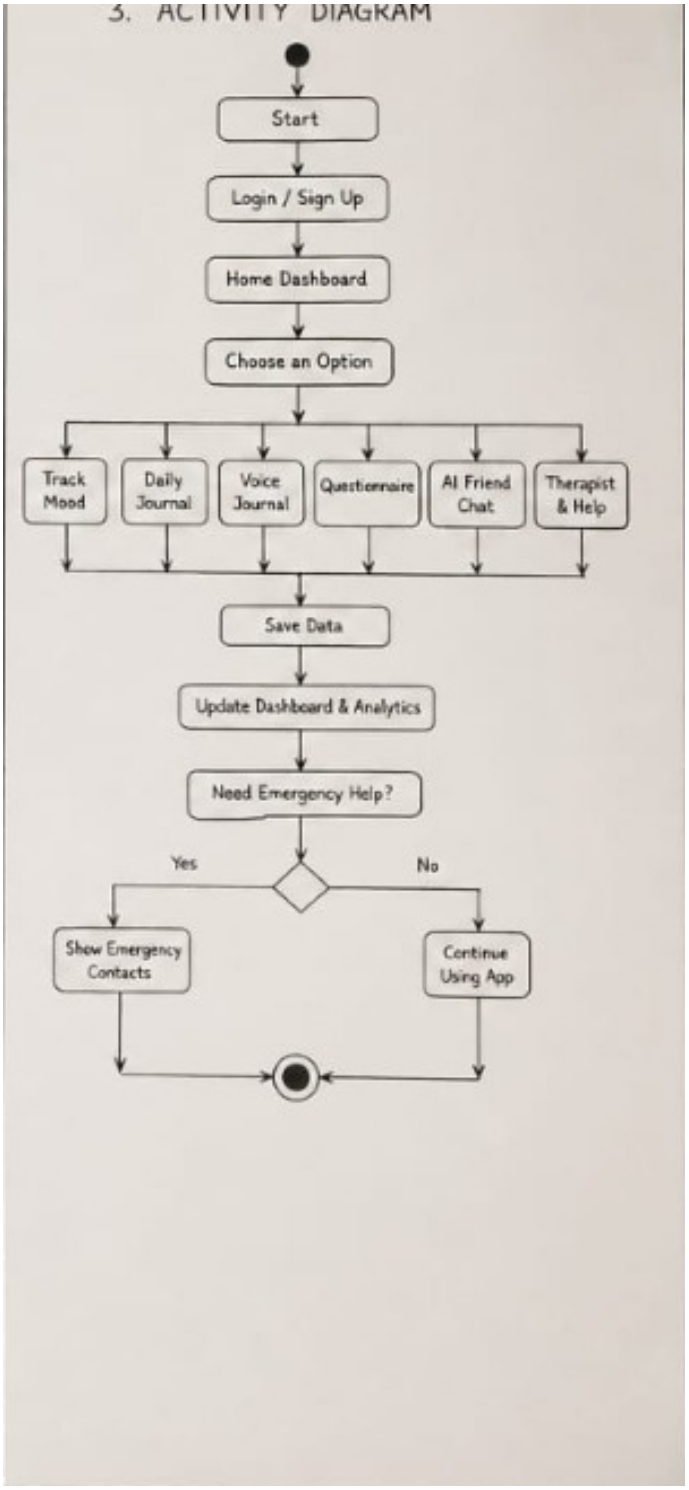


Figure 2.3: Activity Diagram showing Operational Flow and Decision Logic

ASSIGNMENT 2: UML MODELING — 4.4 SEQUENCE DIAGRAM

4.4 Sequence Diagram Chronological Message Flow

The Sequence Diagram depicts the time-ordered message exchange among 5 participating lifelines: User, Web App, Server, Database, and AI Service (OpenAI API).

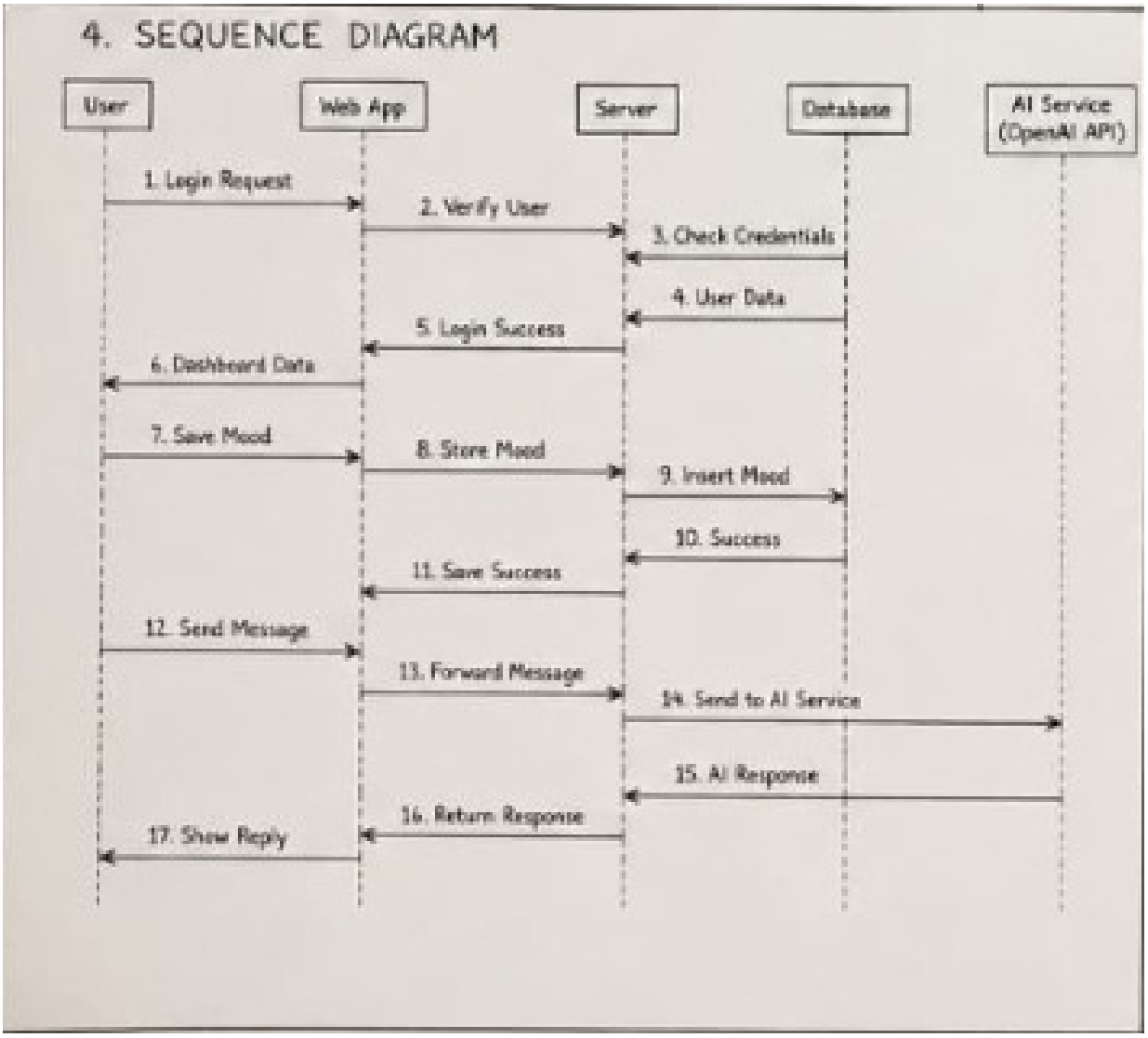


Figure 2.4: Sequence Diagram of Authentication, Mood Logging, and AI Chat

Phase / Flow	Message Sequence (1 to 17)
1. Authentication	1. Login Request $\rightarrow$ 2. Verify User $\rightarrow$ 3. Check Credentials $\rightarrow$ 4. User Data $\rightarrow$ 5. Login Success $\rightarrow$ 6. Dashboard Data returned to User.
2. Save Mood	7. Save Mood $\rightarrow$ 8. Store Mood $\rightarrow$ 9. Insert Mood in DB $\rightarrow$ 10. Success $\rightarrow$ 11. Save Success returned to User.
3. AI Interaction	12. Send Message $\rightarrow$ 13. Forward Message $\rightarrow$ 14. Send to AI Service $\rightarrow$ 15. AI Response $\rightarrow$ 16. Return Response $\rightarrow$ 17. Show Reply in Chat UI.

ASSIGNMENT 2: UML MODELING — 4.5 COMPONENT DIAGRAM

4.5 Component Diagram Architectural Tiers

The Component Diagram visualizes the physical breakdown of the software into autonomous, loosely coupled modules across client, application, database, and cloud service layers.

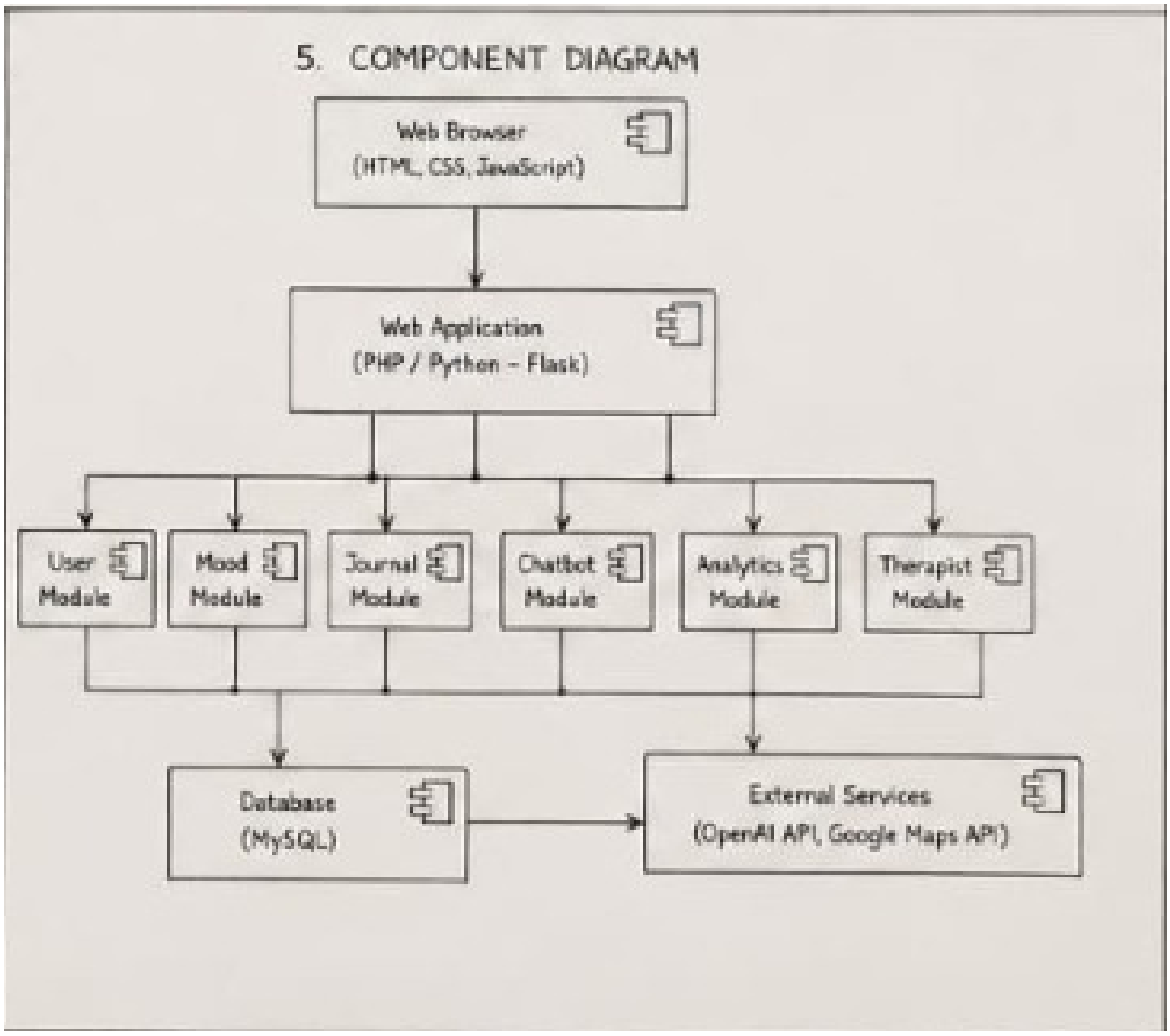


Figure 2.5: Component Diagram showcasing Modular Multi-Tier Architecture

Architectural Layer	Components & Sub-Modules Included	Primary Responsibility
Client Tier	Web Browser (HTML5, CSS3, JavaScript)	Renders UI, handles audio recording and forms.
Application Tier	Web Application (PHP / Python Flask): • User Module • Mood Module • Journal Module • Chatbot Module • Analytics Module • Therapist Module	Executes business logic, authenticates requests, processes analytics, and connects to data/APIs.
Data & Cloud Tier	• Database (MySQL) • External APIs (OpenAI, Google Maps)	Persists application state; provides AI natural language reasoning and therapist geolocation.

ASSIGNMENT 3: SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)

1. What is Project Life Cycle and its Main Characteristics?

Definition: The Project Life Cycle encompasses the formal sequence of phases through which a software project transitions from inception to project close: **Initiation $\rightarrow$ Planning $\rightarrow$ Execution $\rightarrow$ Monitoring & Controlling $\rightarrow$ Project Closure**.

Main Characteristics:

1. **Distinct Phase Boundaries:** Every phase features explicit prerequisites, entrance criteria, milestone work products, and exit sign-offs.
2. **Staffing & Expenditure Curve:** Costs and human resources begin low, reach peak intensity during coding and integration, and drop rapidly toward delivery.
3. **Uncertainty & Risk Profile:** Risk and ambiguity are at their highest at project initiation and decrease progressively as requirements and architectures freeze.
4. **Cost of Modification:** Altering specifications is inexpensive early in planning, but becomes exponentially costly once coding, testing, and deployment commence.

2. Explain the following SDLC models:

i) Waterfall Model

A linear sequential SDLC model where development proceeds monotonically downward through distinct phases: Requirements $\rightarrow$ Design $\rightarrow$ Implementation $\rightarrow$ Testing $\rightarrow$ Deployment $\rightarrow$ Maintenance. Each phase must finish completely before the next commences.

- *Strengths:* High structure, simple milestone tracking, and extensive formal documentation.
- *Weaknesses:* Extremely rigid; late defect discovery; impossible to accommodate shifting requirements.
- *Best Application:* Small projects with fixed scope and thoroughly understood requirements.

ii) Iterative Model

Develops a partial working baseline of the system first and incrementally enhances functionality across repeated cyclical iterations until the full system is completed.

- *Strengths:* Delivers working software early; enables stakeholder feedback to shape subsequent cycles.
- *Weaknesses:* Demands vigilant architecture governance; risk of feature creep and scope ballooning.
- *Best Application:* Large-scale software systems where fundamental objectives are defined but details evolve.

iii) Spiral Model

A risk-driven meta-model combining iterative prototyping with Waterfall discipline. It progresses through 4 repeating quadrants: (1) Determine Objectives, (2) Identify & Resolve Risks via prototypes, (3) Develop & Verify, and (4) Plan Next Iteration.

- *Strengths:* World-class risk management; prevents disastrous project failures.
- *Weaknesses:* Costly; demands rare risk assessment expertise; unsuitable for small or low-budget projects.
- *Best Application:* Mission-critical, high-uncertainty, large enterprise software.

iv) V-shaped Model (Verification & Validation)

An extension of the Waterfall model where each development (Verification) phase is executed with a corresponding testing (Validation) phase planned in parallel:

  – Requirements Analysis $\leftrightarrow$ Acceptance Testing
  – System Architecture Design $\leftrightarrow$ System Testing
  – Module / Component Design $\leftrightarrow$ Integration Testing
  – Coding $\leftrightarrow$ Unit Testing

- *Strengths:* Early test case design; high discipline; excellent bug prevention.
- *Weaknesses:* Inflexible to mid-project requirement revisions; lacks early prototype capability.
- *Best Application:* High-integrity systems (e.g., medical devices, avionics, defense software).

v) Agile Model

A collaborative, iterative, and incremental philosophy based on the Agile Manifesto. It delivers functional software in short, timeboxed sprints (1–4 weeks), emphasizing customer collaboration, adaptability, and working software over rigid documentation.

- *Strengths:* Highly responsive to changing client needs; rapid time-to-market; continuous feedback.
- *Weaknesses:* Less formal documentation; requires high client commitment; scope can expand without strict backlog grooming.
- *Best Application:* Dynamic modern web/mobile applications with evolving user requirements.

ASSIGNMENT 4: SOFTWARE TESTING, SCM & QUALITY ASSURANCE

1] Define Software Testing?

Software Testing is the systematic engineering process of executing a software application under specified conditions to uncover bugs and defects, verify that the system satisfies all documented functional and non-functional requirements, and validate that it fulfills user needs and quality benchmarks.

2] What is Risk identification?

Risk identification is the proactive initial stage of risk management wherein project, technical, operational, and commercial threats (such as scope churn, unrealistic deadlines, key developer attrition, security flaws, and third-party service downtime) are systematically identified, categorized, and documented in a risk register.

3] What is SCM (Software Configuration Management)?

SCM is an engineering discipline that controls, tracks, and manages modifications to software artifacts (source code, build scripts, requirements, database schemas, test cases) across the SDLC. It ensures traceability, repeatable releases, and system integrity across software versions.

4] Define Debugging.

Debugging is the diagnostic problem-solving process of detecting, isolating, analyzing, and resolving the underlying root cause of a defect, crash, or bug discovered during software testing or production execution.

5] Explain Configuration Audit.

A Configuration Audit is an independent review to ensure that a configuration baseline meets all specified requirements and is ready for deployment. It comprises two audits:

• **Functional Configuration Audit (FCA):** Verifies that the software product functions strictly in accordance with its software requirement specifications (SRS).

• **Physical Configuration Audit (PCA):** Verifies that all required files, documentation, release notes, and configuration artifacts are physically present in the release package.

6] Differentiate Between Black Box Testing and White Box Testing?

Comparison Parameter	Black Box Testing	White Box Testing
Core Focus	Validates external software behavior and compliance with functional specifications.	Validates internal code logic, data flow, structure, and branch paths.
Code Visibility	Source code is hidden; tester acts as an end user.	Source code is completely visible and scrutinized.
Performed By	Independent Software Testers, QA Engineers, and End Users.	Software Developers and specialized Automation Engineers.
Programming Knowledge	Not required.	Mandatory (deep knowledge of code syntax and structure).
Primary Techniques	Equivalence Partitioning, Boundary Value Analysis (BVA), Decision Tables, State Transition.	Statement Coverage, Branch/Decision Coverage, Condition Coverage, Path Coverage.
Typical Testing Level	System Testing, Functional Testing, Acceptance Testing (UAT).	Unit Testing, Component Testing, Integration Testing.

ASSIGNMENT 4: SOFTWARE TESTING, SCM & QUALITY ASSURANCE (CONTINUED)

7] What Do You Mean By Metrics?

Software Metrics are standard quantitative measurements used to assess, track, and optimize the quality, productivity, and performance of software products and development processes.

- **Product Metrics:** Measure software attributes directly (e.g., Lines of Code - LOC, Cyclomatic Complexity, Defect Density).
- **Process Metrics:** Measure development efficiency (e.g., Defect Removal Efficiency - DRE, Mean Time to Detect - MTTD, Lead Time).
- **Project Metrics:** Measure project trajectory (e.g., Schedule Variance, Cost Variance, Test Pass Rate %).

8] What Do You Mean By Version Control?

Version Control (or Source Code Management) is a system (e.g., **Git**, Subversion) that records incremental revisions made to codebases over time. It allows engineering teams to examine audit logs of historical edits, roll back to prior stable commits, compare line-by-line differences, and collaborate concurrently through branching and merging without overwriting each other's work.

9] Explain Object Oriented Software Engineering (OOSE)?

Object Oriented Software Engineering (OOSE) is a development methodology that conceives software systems as networks of interacting, autonomous **objects** encapsulating both internal data attributes and functional behaviors. It leverages four cornerstone Object-Oriented principles:

- **Abstraction:** Exposes only essential interfaces while concealing implementation intricacies.
- **Encapsulation:** Bundles data and methods securely within class boundaries to prevent unauthorized mutation.
- **Inheritance:** Establishes hierarchical relationships enabling code reuse and specialization.
- **Polymorphism:** Enables entities to take on different behaviors dynamically via method overloading and overriding.

OOSE fosters high modularity, high cohesion, low coupling, and enhanced software reusability.

10] Explain Various Types of Testing in details.

Software testing is broadly categorized into **Functional** and **Non-Functional** testing tiers:

1. **Unit Testing:** Verifies the correctness of individual functions, methods, or isolated modules (using frameworks like PyTest, JUnit).
2. **Integration Testing:** Verifies interface compatibility and seamless data transfer between interconnected modules.
3. **System Testing:** Evaluates the complete, integrated software build against the comprehensive business requirements.
4. **Acceptance Testing (UAT):** Executed by end users or clients to confirm the software is ready for production rollout.
5. **Regression Testing:** Re-executing existing test suites after code changes to confirm prior features remain unbroken.
6. **Performance & Load Testing:** Assesses application latency, memory consumption, throughput, and stability under heavy concurrent traffic.
7. **Security Testing:** Identifies vulnerabilities, SQL injection risks, authentication bypasses, and data encryption flaws.

11] Differentiate Between Automated Testing and Manual Testing.

Comparison Parameter	Manual Testing	Automated Testing
Execution Mode	Carried out manually by human testers navigating the application step-by-step.	Executed automatically via test automation scripts, frameworks, and tools.
Execution Speed	Slow, time-consuming, and labor-intensive.	Significantly faster; can run thousands of tests rapidly.
Human Error & Fatigue	Prone to human error, missed steps, and fatigue during repetitive runs.	Zero fatigue; results are consistent, exact, and repeatable.
Cost Efficiency	Lower initial cost; high recurring cost per test execution.	Higher initial setup and tool cost; very low cost per run over time.
Ideal Use Cases	Exploratory testing, UI/UX evaluation, and ad-hoc usability testing.	Regression testing, load/stress testing, and continuous integration (CI/CD).
Popular Tools	Postman (manual API calls), Bugzilla, Jira, TestRail.	Selenium, Playwright, Cypress, JUnit, PyTest, JMeter.